Université Paris Sorbonne Nord
Institut Galilée

Master 1.
Distributed Java Programming
TP2: Exceptions and lambda-expressions

Define a class `Teacher` with 4 fields :

- `ID` (a unique identifier).
- `String name`
- `int age`
- `float salary`
- `Discipline discipline` : an enumeration (`Discipline`) for `CS`, `MATH`, `PHYSICS`, `CHEMISTRY`, `ENGLISH`.

Add constructors, getters and setters. Define also the methods `equals`, `toString`, `hashCode` and `clone`. See the Java Documentation of the class `Object` to understand the contract of these methods and to implement them accordingly.
Now we define functionalities for the whole set of teachers defined in the software. Either you define a class that will work as a container (say `TeacherContainer`), or you add to the class `Teacher` a static field (say `staff`). Choose one of the two options and define methods to add and remove teachers to the database.
Read the documentation of the interface `Stream<T>` and use lambda-expressions (and operations on streams) to implement the following methods/queries.

1. Given the ID of a teacher, return his/her name.
2. Compute the average of teacher's ages.
3. Compute the average of the salaries of teachers whose ages are between 40 and 60
4. Check if all the teachers in the database are older than 30 years.
5. Check if there is a teacher whose salary is above 5.000.
6. Compute the minimum and the maximum salaries. Try to compute both values with the *same* stream. What happens? Here a possible solution to "reuse" the stream[1]:

[1] In java 12, the method `teeing` of the class `Collectors` allows us to do it in a neater way

```
Supplier<DoubleStream> supplier =
            () -> Teacher.stuff.stream() ...
        double min = supplier.get()...
        double max = supplier.get()....
```

7. Compute the average of the salaries, but grouping the results by the discipline of the teacher (see the method `collect` of the interface `Stream`).

Write a `main` method to test your classes.